Step 1:Import Python Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime

path = "C:\\Users\\sanrkin\\Downloads\\used_cars_data.csv"
df_used_cars = pd.read_csv(path)
```

Step 2:Overview of the dataset

```
print("Shape of the dataset:", df_used_cars.shape)
print(f"Rows: {df_used_cars.shape[0]}")
print(f"Columns: {df_used_cars.shape[1]}")

# First 5 rows (quick preview)
df_used_cars.head(5)

# Last 5 rows (quick preview)
df_used_cars.tail(5)

# Dataset summary (columns, types, missing values)
df_used_cars.info()

# Stats summary (mean, min, max, etc.)
df_used_cars.describe()

# Unique values in each column
print(df_used_cars.nunique())
```

Step 3: Handling Missing & Duplicate Data

```
df_used_cars.isnull().sum()

# Check the percentage of missing values in each column
print(df_used_cars.isnull().sum()/len(df_used_cars)*100)
```

The dataset contains missing values. The "New Price" column has the highest number of null values, The "Price" column has the null values (17.01365%). Other features like "Seats" (0.730732%), "Engine" (0.634220%), "Power" (0.634220%), and "Mileage" (0.027575%) also contain null values, though to a lesser extent.

```
# Drop the "New-price" column it has most number of missing values
df_used_cars = df_used_cars.drop("New_Price",axis=1)
```

```
df_used_cars.info()

# Duplicate records
df_used_cars.duplicated().sum()
```

Step 4: Plot the features

```
plt.figure(figsize=(15,10))

plt.subplot(2,2,1)
sns.histplot(df_used_cars['Mileage'],kde=True)
plt.title('Mileage Distribution')

plt.subplot(2,2,2)
sns.histplot(df_used_cars['Engine'],kde=True)
plt.title('Engine Distribution')

plt.subplot(2,2,3)
sns.histplot(df_used_cars['Power'],kde=True)
plt.title('Power Distribution')

plt.subplot(2,2,4)
sns.histplot(df_used_cars['Price'],kde=True)
plt.title('Price Distribution')

# Adjust the spacing between subplots to prevent overlap of titles
plt.tight_layout()
plt.show()
```

Mileage: Positively skewed distribution (skewed right). Most vehicles cluster in lower mileage range, with a long tail extending toward high mileage values.

Engine: Distribution with positive skewness. Shows two distinct peaks for engine sizes, with a longer tail toward larger engines.

Power: Strong positive skewness (skewed right). Concentrated in lower power ratings, with a long stretched tail toward high-power vehicles.

Price: Highly positively skewed (skewed right). Heavy concentration in lower price range ($10-20K), with an extensive right tail showing fewer but progressively more expensive vehicles.

```
df_used_cars = df_used_cars.dropna(subset=['Price'])

df_used_cars['Mileage'] = df_used_cars['Mileage'].astype(str)

# Extract numeric values from 'Mileage' and convert to float
df_used_cars['Mileage'] = df_used_cars['Mileage'].str.extract('(\d+\.\
d+|\d+)')[0].astype(float)
```

```
# Fill missing values with the median of 'Mileage'
df_used_cars['Mileage'] =
df_used_cars['Mileage'].fillna(df_used_cars['Mileage'].median())
```

The values in the 'Mileage' column contain entries like '18 kmpl', '20.5 kmpl', and '14.2 km/kg', so we converted them to strings and extracted the numeric values.

```
df_used_cars['Engine'] = df_used_cars['Engine'].astype(str)
df_used_cars['Engine'] = df_used_cars['Engine'].str.extract('(\d+)')
[0].astype(float)
df_used_cars['Engine'] =
df_used_cars['Engine'].fillna(df_used_cars['Engine'].median())
df_used_cars['Engine'] = df_used_cars['Engine'].astype('int64')


df_used_cars['Power'] = df_used_cars['Power'].astype(str)

# Extract numeric values from 'Power' and convert to float
df_used_cars['Power'] = df_used_cars['Power'].str.extract('(\d+\.\d+|\
d+)')[0].astype(float)

# Fill missing values with the median of 'Power'
df_used_cars['Power'] =
df_used_cars['Power'].fillna(df_used_cars['Power'].median())

df_used_cars['Seats'] =
df_used_cars['Seats'].fillna(df_used_cars['Seats'].mode()[0])

# check if the dataset contain null values
df_used_cars.isnull().sum()

# Age of a car

current_year = datetime.now().year
df_used_cars['Car_age'] = current_year - df_used_cars['Year']

df_used_cars.head(5)
```

Step 5: EDA Univariate Analysis

```


df_used_cars.info()

numerical_columns = df_used_cars.select_dtypes(include='number')
numerical_columns.columns

# Get categorical columns in the DataFrame
categorical_columns = df_used_cars.select_dtypes(include=['object',
'category'])
categorical_columns.columns
```

Step 6: Removing the outliers

```
numerical_df = df_used_cars.select_dtypes(include=['float64',
'int64'])

# Correlation Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(numerical_df.corr(), annot=True, cmap='coolwarm',
linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()

# Define important numerical columns for car price analysis
features = ['Kilometers_Driven', 'Mileage', 'Engine', 'Power',
'Price']

print("Analyzing these Features:", features)

# Create boxplots before removing outliers
plt.figure(figsize=(12, 6))
sns.boxplot(data=df_used_cars[features],color='lightcoral')
plt.title('Boxplots Before Outlier Removal')
plt.show()

# Function to remove outliers using IQR method
def remove_outliers(df, columns):
    df_filtered = df.copy()  # Create a copy to avoid modifying the
original DataFrame
    for col in columns:
        Q1 = df_filtered[col].quantile(0.25)
        Q3 = df_filtered[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df_filtered = df_filtered[(df_filtered[col] >= lower_bound) &
(df_filtered[col] <= upper_bound)]
    return df_filtered

# Remove outliers from selected columns
df_cleaned = remove_outliers(df_used_cars, features)


# Remove outliers from selected columns
df_cleaned = remove_outliers(df_used_cars, features)

# Create boxplots after outlier removal
plt.figure(figsize=(12, 6))
sns.boxplot(data=df_cleaned[features],color='lightcoral')
plt.title('Boxplots After Outlier Removal')
plt.show()
```

```
# Comparison dataset shapes
print("Original dataset shape:", df_used_cars.shape)
print("Cleaned dataset shape:", df_cleaned.shape)
rows_removed = df_used_cars.shape[0] - df_cleaned.shape[0]
print(f"Removed {rows_removed} rows")
print(f"Percentage of data removed: {rows_removed /
df_used_cars.shape[0] * 100:.2f}%")

#Feature Analysis
#Numerical Features Analysis
# Plot distributions before and after outlier removal
for col in features:
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    sns.histplot(df_used_cars[col], kde=True,color='skyblue')
    plt.title(f'{col} Before Outlier Removal')

    plt.subplot(1, 2, 2)
    sns.histplot(df_cleaned[col], kde=True,color='skyblue')
    plt.title(f'{col} After Outlier Removal')

    plt.tight_layout()
    plt.show()

# Categorical columns
categorical_cols = ['Location', 'Fuel_Type', 'Transmission',
'Owner_Type']

sns.set_style("whitegrid")

# Categorical features with charts and percentages
for col in categorical_cols:
    plt.figure(figsize=(10, 4))
    sns.countplot(y=df_cleaned[col],
order=df_cleaned[col].value_counts().index)
    plt.title(f'{col} Distribution')
    plt.grid(axis='x', linestyle='-', alpha=0.7)
    plt.show()

    percentages = (df_cleaned[col].value_counts() / len(df_cleaned)) *
100
    percentages = percentages.round(2)

    print(f"\n{col} Percentages:")
    print(percentages)

# Scatter plots for Price vs Numerical Columns
numerical_cols = ['Kilometers_Driven', 'Mileage', 'Engine', 'Power']
```

```
for col in numerical_cols:
    plt.figure(figsize=(10, 6))
    sns.scatterplot(data=df_cleaned, x=col, y='Price')
    plt.title(f'Price vs {col}')
    plt.show()
```

1.Price vs Engine Relationship

- Strong positive correlation observed between engine size and price
- Price variation increases with larger engine sizes

2.Price vs Mileage Analysis

- No significant correlation between mileage and price
- Wide price distribution across all mileage values

1. Price vs Kilometers_Driven Analysis

- Weak negative correlation detected
- Higher concentration of data points in lower kilometer range
- Price variability decreases as kilometers driven increases

```
# Boxplot for Price vs Categorical Columns
categorical_cols = ['Location', 'Fuel_Type', 'Transmission',
'Owner_Type']

for col in categorical_cols:
    plt.figure(figsize=(10, 6))
    sns.boxplot(x=df_cleaned[col], y=df_cleaned['Price'])
    plt.title(f'Price vs {col}')
    plt.show()
```

1.Price vs Owner Type:

Clear price depreciation pattern: First owner vehicles have highest prices, followed by steady decrease through subsequent owners First-hand cars show significantly higher median prices compared to others

2.Price vs Fuel Type:

Electric and Diesel vehicles command highest prices in the market CNG and LPG vehicles fall in the most affordable price range Petrol vehicles occupy the middle price range with high variability

3.Price vs Location:

Significant price variations across cities with Coimbatore, Mumbai, and Delhi showing higher median prices Tier-2 cities generally show lower price ranges All locations show outliers in premium segments, indicating presence of luxury market across cities